

# Hibernate - Master Notes

A Complete Reference for Interview Preparation, Production Work, Real-World Projects, and System Understanding.

**Java Full Stack Master Course**

# Table of Contents

|                                                                                                   |           |
|---------------------------------------------------------------------------------------------------|-----------|
| <b>HIBERNATE — MASTER NOTES</b>                                                                   | <b>6</b>  |
| A Complete Reference Book for Interviews, Production Work & System Understanding . . . . .        | 6         |
| Written for someone who doesn't yet know WHY Hibernate exists or what problem it solves . . . . . | 6         |
| <b>TOPIC 1: ORM (OBJECT-RELATIONAL MAPPING)</b>                                                   | <b>6</b>  |
| 1. Introduction . . . . .                                                                         | 6         |
| 1.1 What is it? . . . . .                                                                         | 6         |
| 1.2 Why was it introduced — The EXACT Problem It Solves . . . . .                                 | 6         |
| 1.3 The "Impedance Mismatch" Problem (The Deeper, Theoretical Reason) . . . . .                   | 7         |
| 1.4 Interview Definition . . . . .                                                                | 8         |
| 1.5 Hibernate's Relationship to JPA — A Critical Distinction . . . . .                            | 8         |
| 2. Real Life Analogy . . . . .                                                                    | 8         |
| 3. Internal Working — What Actually Happens . . . . .                                             | 9         |
| 4. Setting Up Hibernate — Dependencies and Configuration . . . . .                                | 9         |
| Maven Dependency . . . . .                                                                        | 9         |
| hibernate.cfg.xml — Basic Configuration . . . . .                                                 | 9         |
| 5. Code Example — The FULL Before/After Comparison . . . . .                                      | 10        |
| 6. Common Mistakes . . . . .                                                                      | 10        |
| 7. Best Practices . . . . .                                                                       | 10        |
| 8. Production Usage . . . . .                                                                     | 10        |
| 9. Frequently Asked Interview Questions . . . . .                                                 | 11        |
| <b>TOPIC 2: SESSIONFACTORY</b>                                                                    | <b>11</b> |
| 1. Introduction . . . . .                                                                         | 11        |
| 2. Code Example — Building a SessionFactory . . . . .                                             | 12        |
| 3. Real Life Analogy . . . . .                                                                    | 12        |
| 4. Frequently Asked Interview Questions . . . . .                                                 | 12        |
| <b>TOPIC 3: SESSION</b>                                                                           | <b>12</b> |
| 1. Introduction . . . . .                                                                         | 13        |
| 2. SessionFactory vs Session — Comparison Table . . . . .                                         | 13        |

|                                                                                         |    |
|-----------------------------------------------------------------------------------------|----|
| 3. Code Example — The Core CRUD Operations . . . . .                                    | 13 |
| 4. "Dirty Checking" — Why updateSalary() Above Needed NO Explicit Update Call . . . . . | 14 |
| 5. The Four Entity States (Object Lifecycle) . . . . .                                  | 14 |
| 6. Real Life Analogy . . . . .                                                          | 14 |
| 7. Edge Cases . . . . .                                                                 | 14 |
| 8. Common Mistakes . . . . .                                                            | 14 |
| 9. Frequently Asked Interview Questions . . . . .                                       | 15 |

## **TOPIC 4: ENTITY MAPPING** **15**

|                                                                         |    |
|-------------------------------------------------------------------------|----|
| 1. Introduction . . . . .                                               | 15 |
| 2. Core Mapping Annotations — Full Reference . . . . .                  | 16 |
| 3. GenerationType Strategies . . . . .                                  | 16 |
| 4. @Enumerated — Why EnumType.STRING Is Almost Always Correct . . . . . | 17 |
| 5. Real Life Analogy . . . . .                                          | 17 |
| 6. Edge Cases . . . . .                                                 | 17 |
| 7. Common Mistakes . . . . .                                            | 17 |
| 8. Frequently Asked Interview Questions . . . . .                       | 17 |

## **TOPIC 5: RELATIONSHIPS** **18**

|                                                                                     |    |
|-------------------------------------------------------------------------------------|----|
| 1. Introduction . . . . .                                                           | 18 |
| 2. @ManyToOne — Many Employees, One Department . . . . .                            | 18 |
| 3. @OneToMany — One Department, Many Employees (The Inverse Side) . . . . .         | 18 |
| 4. @OneToOne — One Employee, One Parking Spot . . . . .                             | 19 |
| 5. @ManyToMany — Employees and Projects (Many-to-Many Needs a Join Table) . . . . . | 19 |
| 6. Full CRUD Example — Saving Related Entities . . . . .                            | 20 |
| 7. Real Life Analogy . . . . .                                                      | 20 |
| 8. Edge Cases . . . . .                                                             | 20 |
| 9. Common Mistakes . . . . .                                                        | 20 |
| 10. Best Practices . . . . .                                                        | 21 |
| 11. Frequently Asked Interview Questions . . . . .                                  | 21 |

## **TOPIC 6: FETCH TYPES** **21**

|                           |    |
|---------------------------|----|
| 1. Introduction . . . . . | 21 |
|---------------------------|----|

|                                                                           |    |
|---------------------------------------------------------------------------|----|
| 2. Code Example — Specifying Fetch Type . . . . .                         | 22 |
| 3. Default Fetch Types (An Important, Frequently-Tested Detail) . . . . . | 22 |
| 4. The Trade-off — Why LAZY Isn't Always "Better" . . . . .               | 22 |
| 5. Real Life Analogy . . . . .                                            | 22 |
| 6. Frequently Asked Interview Questions . . . . .                         | 22 |

## **TOPIC 7: CASCADE TYPES 23**

|                                                             |    |
|-------------------------------------------------------------|----|
| 1. Introduction . . . . .                                   | 23 |
| 2. Code Example . . . . .                                   | 23 |
| 3. The Cascade Types — Full Reference . . . . .             | 23 |
| 4. orphanRemoval — A Related but DISTINCT Concept . . . . . | 23 |
| 5. Real Life Analogy . . . . .                              | 23 |
| 6. Edge Cases . . . . .                                     | 24 |
| 7. Common Mistakes . . . . .                                | 24 |
| 8. Frequently Asked Interview Questions . . . . .           | 24 |

## **TOPIC 8: HQL (HIBERNATE QUERY LANGUAGE) 24**

|                                                   |    |
|---------------------------------------------------|----|
| 1. Introduction . . . . .                         | 25 |
| 2. HQL vs SQL — The Key Difference . . . . .      | 25 |
| 3. Code Examples — Common HQL Patterns . . . . .  | 25 |
| 4. Real Life Analogy . . . . .                    | 25 |
| 5. Edge Cases . . . . .                           | 26 |
| 6. Common Mistakes . . . . .                      | 26 |
| 7. Frequently Asked Interview Questions . . . . . | 26 |

## **TOPIC 9: CRITERIA API 26**

|                                                                    |    |
|--------------------------------------------------------------------|----|
| 1. Introduction . . . . .                                          | 26 |
| 2. Why It Exists — The Problem With String-Based Queries . . . . . | 26 |
| 3. Code Example — Building a Dynamic Query . . . . .               | 27 |
| 4. HQL vs Criteria API — When to Use Which . . . . .               | 27 |
| 5. Frequently Asked Interview Questions . . . . .                  | 27 |

## **TOPIC 10: CACHING 28**

|                                                                         |    |
|-------------------------------------------------------------------------|----|
| 1. Introduction . . . . .                                               | 28 |
| 2. First-Level Cache (L1) — Automatic, Per-Session, Always On . . . . . | 28 |
| 3. Second-Level Cache (L2) — Optional, Shared ACROSS Sessions . . . . . | 28 |
| 4. Real Life Analogy . . . . .                                          | 28 |
| 5. Edge Cases . . . . .                                                 | 29 |
| 6. Common Mistakes . . . . .                                            | 29 |
| 7. Frequently Asked Interview Questions . . . . .                       | 29 |

## **TOPIC 11: LAZY VS EAGER LOADING (DEEP DIVE) 29**

|                                                                                                 |    |
|-------------------------------------------------------------------------------------------------|----|
| 1. Introduction . . . . .                                                                       | 29 |
| 2. LazyInitializationException — The #1 Real-World Hibernate Error . . . . .                    | 30 |
| 3. Why This Happens — The Proxy Mechanism . . . . .                                             | 30 |
| 4. The Three Standard Fixes . . . . .                                                           | 30 |
| Fix 1: Fetch it EAGERLY (simplest, but loses lazy loading's benefits for that field) . . . . .  | 30 |
| Fix 2: Access it WHILE the Session is still open (the recommended, most flexible fix) . . . . . | 30 |
| Fix 3: Use JOIN FETCH in HQL/JPQL to load everything in ONE query upfront . . . . .             | 30 |
| 5. The "N+1 Select Problem" — A Related, Critical Performance Issue . . . . .                   | 31 |
| 6. Real Life Analogy . . . . .                                                                  | 31 |
| 7. Edge Cases . . . . .                                                                         | 31 |
| 8. Common Mistakes . . . . .                                                                    | 31 |
| 9. Best Practices . . . . .                                                                     | 32 |
| 10. Production Usage . . . . .                                                                  | 32 |
| 11. Frequently Asked Interview Questions . . . . .                                              | 32 |
| 12. Revision Notes (Entire Hibernate Section Consolidated) . . . . .                            | 33 |
| 13. Homework (Covers Entire Hibernate Section) . . . . .                                        | 33 |
| 14. Mini Project — A Simple Employee Management DAO Layer . . . . .                             | 34 |
| FINAL SUMMARY — HIBERNATE COMPLETE . . . . .                                                    | 34 |

# HIBERNATE — MASTER NOTES

*A Complete Reference Book for Interviews, Production Work & System Understanding*

*Written for someone who doesn't yet know WHY Hibernate exists or what problem it solves*

---

## TOPIC 1: ORM (OBJECT-RELATIONAL MAPPING)

---

### 1. Introduction

#### 1.1 What is it?

ORM (Object-Relational Mapping) is a technique/library that automatically translates between two fundamentally different worlds:

- **Java's world:** objects, classes, references, inheritance.
- **The database's world:** tables, rows, columns, foreign keys.

**Hibernate** is the most widely-used ORM framework for Java — it lets you work with plain Java objects (`Employee employee = new Employee();`) and automatically generates and executes the correct SQL behind the scenes to save, load, update, and delete that data, WITHOUT you hand-writing that SQL yourself for routine operations.

#### 1.2 Why was it introduced — The EXACT Problem It Solves

Let's look at what life was like using **raw JDBC** (which you learned in the JDBC section) to do something as simple as "save an Employee object to the database":

```
// THE "BEFORE HIBERNATE" WORLD - raw JDBC, for ONE simple save operation
public void saveEmployee(Employee emp) throws SQLException {
    String sql = "INSERT INTO employees (first_name, last_name, dept_id, salary, hire_date) VALUES (?, ?, ?, ?, ?)";
    try (Connection conn = dataSource.getConnection();
        PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setString(1, emp.getFirstName());
        pstmt.setString(2, emp.getLastName());
        pstmt.setInt(3, emp.getDeptId());
        pstmt.setBigDecimal(4, emp.getSalary());
        pstmt.setDate(5, java.sql.Date.valueOf(emp.getHireDate()));
        pstmt.executeUpdate();
    }
}

// And to READ it back, you need ANOTHER large block of code:
public Employee findEmployee(int id) throws SQLException {
    String sql = "SELECT * FROM employees WHERE emp_id = ?";
    try (Connection conn = dataSource.getConnection();
        PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setInt(1, id);
        try (ResultSet rs = pstmt.executeQuery()) {
```

```

if (rs.next()) {
    Employee emp = new Employee();
    emp.setEmpId(rs.getInt("emp_id"));
    emp.setFirstName(rs.getString("first_name"));
    emp.setLastName(rs.getString("last_name"));
    emp.setDeptId(rs.getInt("dept_id"));
    emp.setSalary(rs.getBigDecimal("salary"));
    emp.setHireDate(rs.getDate("hire_date").toLocalDate());
    return emp;
}
}
return null;
}

```

Now imagine you have **50 different entity types** (Employee, Department, Order, Product, Customer, ...) in a real enterprise application. You'd need to hand-write THIS SAME REPETITIVE boilerplate — manually mapping every single column to every single field, in BOTH directions — for every single one of them. This is:

- **Extremely repetitive** (the exact same "set field from ResultSet column" pattern, over and over).
- **Error-prone** (typo a column name in one of 50 classes, get a runtime bug).
- **Hard to maintain** (add ONE new field to `Employee`, and you must remember to update the `INSERT`, the `SELECT`-mapping code, the `UPDATE` — potentially in MULTIPLE places).
- **Manually manages relationships** (loading an `Employee` AND their `Department` requires manually writing a `JOIN` or a second query, then manually stitching the objects together).

**Hibernate eliminates ALL of this boilerplate.** You define your class ONCE with simple annotations, and Hibernate automatically generates the correct SQL, handles the object ↔ row translation, AND manages relationships between objects — for every single entity, consistently, without you writing that repetitive mapping code by hand ever again.

### 1.3 The "Impedance Mismatch" Problem (The Deeper, Theoretical Reason)

There's a formal name for why this translation is genuinely hard, not just "annoying": the **Object-Relational Impedance Mismatch** — Java's object model and the relational model fundamentally think about data differently:

| Concept       | Object-Oriented World (Java)                                                               | Relational World (SQL Database)                                                                             |
|---------------|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Identity      | An object's identity is its memory reference                                               | A row's identity is its primary key value                                                                   |
| Relationships | An object HOLDS a reference to another object<br>( <code>employee.getDepartment()</code> ) | A row references another row via a foreign key VALUE, requiring a JOIN to actually retrieve it              |
| Inheritance   | Java supports class inheritance<br>( <code>Manager extends Employee</code> )               | Relational tables have NO native concept of inheritance — must be simulated via specific mapping strategies |
| Data types    | Rich types: <code>List</code> , <code>Optional</code> , custom classes, enums              | A more limited, standardized set of column types                                                            |

Hibernate's entire job is to BRIDGE this mismatch — automatically and consistently — so YOU get to think in terms of clean Java objects and relationships, while Hibernate handles translating that into the correct relational SQL underneath.

## 1.4 Interview Definition

"ORM (Object-Relational Mapping) is a programming technique that maps object-oriented domain models onto relational database structures, automating the translation between Java objects and database rows/columns; Hibernate is the dominant Java ORM implementation, and also serves as the reference implementation underlying the JPA (Java Persistence API) specification."

## 1.5 Hibernate's Relationship to JPA — A Critical Distinction

- **JPA (Java Persistence API)** is a SPECIFICATION (a set of interfaces/annotations like `@Entity`, `@Id`, `EntityManager`) — it defines WHAT an ORM for Java should look like, but provides NO actual implementation itself.
- **Hibernate** is the most popular CONCRETE IMPLEMENTATION of that JPA specification (other implementations exist too, like EclipseLink).
- You can write code using ONLY standard `jakarta.persistence.*` (JPA) annotations/interfaces, and Hibernate will work underneath — OR you can use Hibernate-SPECIFIC extra features (like HQL, covered later) that go beyond the JPA standard.

```
JPA (the SPECIFICATION / interface / contract)
|
v
Hibernate (the IMPLEMENTATION - what actually runs)
```

## 2. Real Life Analogy

- **Bank:** Before ORM, a bank clerk had to MANUALLY translate every single customer conversation ("I want to open an account") into precise, low-level paperwork forms (raw SQL) by hand, every single time, for every different type of banking product. An ORM is like having an intelligent AUTOMATIC TRANSLATOR/ASSISTANT that instantly converts the clerk's simple, natural request ("save this customer") into all the correct, detailed paperwork automatically — the clerk just deals with customers (objects) naturally, never touching the raw forms (SQL) directly for routine work.
- **Restaurant:** JDBC is like a chef who must personally write out, from scratch, the EXACT low-level cooking instructions for every single dish, every single time. Hibernate is like having a smart kitchen assistant who, once you've described a dish's RECIPE ONCE (the entity mapping), automatically handles preparing it correctly every time it's ordered, without the chef re-writing the low-level steps repeatedly.



### 3. Internal Working — What Actually Happens

```
YOUR JAVA CODE:
Employee emp = new Employee("Alice", 75000);
session.save(emp);
|
v
+-----+
| HIBERNATE |
| 1. Reads the @Entity mapping metadata for Employee |
| 2. GENERATES the correct SQL INSERT statement |
| 3. Uses JDBC underneath (PreparedStatement) to execute |
| that generated SQL against the actual database |
| 4. Manages the object's STATE (tracking whether it's |
| new/dirty/deleted) via its "Persistence Context" |
+-----+
|
v
ACTUAL DATABASE (PostgreSQL, MySQL, etc.)
INSERT INTO employees (first_name, salary) VALUES ('Alice', 75000);
```

**Critical insight:** Hibernate does NOT replace JDBC — it USES JDBC underneath, generating and executing the SQL FOR you, so you don't have to write it by hand for routine, everyday operations.

---

### 4. Setting Up Hibernate — Dependencies and Configuration

#### *Maven Dependency*

```
<dependency>
<groupId>org.hibernate.orm</groupId>
<artifactId>hibernate-core</artifactId>
<version>6.4.0.Final</version>
</dependency>
<dependency>
<groupId>org.postgresql</groupId>
<artifactId>postgresql</artifactId>
<version>42.7.1</version>
</dependency>
```

#### hibernate.cfg.xml — *Basic Configuration*

```
<hibernate-configuration>
<session-factory>
<property name="hibernate.connection.driver_class">org.postgresql.Driver</property>
<property name="hibernate.connection.url">jdbc:postgresql://localhost:5432/company_db</property>
<property name="hibernate.connection.username">postgres</property>
<property name="hibernate.connection.password">yourpassword</property>
<property name="hibernate.dialect">org.hibernate.dialect.PostgreSQLDialect</property>
<property name="hibernate.hbm2ddl.auto">update</property> <!-- auto-creates/updates tables from
entities -->
<property name="hibernate.show_sql">true</property> <!-- prints generated SQL to console - great for
learning! -->
</session-factory>
</hibernate-configuration>
```

## 5. Code Example — The FULL Before/After Comparison

```
// STEP 1: Define your entity ONCE with simple annotations
import jakarta.persistence.*;

@Entity
@Table(name = "employees")
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "first_name")
    private String firstName;

    private Double salary;

    // getters and setters...
}

// STEP 2: NOW saving/loading is THIS simple, for EVERY entity, forever:
Employee emp = new Employee();
emp.setFirstName("Alice");
emp.setSalary(75000.0);

session.persist(emp); // Hibernate generates + runs the INSERT automatically
Employee loaded = session.get(Employee.class, emp.getId()); // Hibernate generates + runs the SELECT
// automatically
System.out.println(loaded.getFirstName()); // "Alice" - NO manual ResultSet mapping code needed!
```

## 6. Common Mistakes

- Assuming Hibernate ELIMINATES the need to understand SQL/JDBC — it doesn't; you still need to understand what's happening underneath (and Hibernate CAN generate inefficient SQL if entities/relationships are mapped carelessly, covered later in Fetch Types/Caching).
- Using `hibernate.hbm2ddl.auto=update` (or worse, `create`) in a PRODUCTION environment — appropriate for learning/development, but risky in production (should use proper migration tools like Flyway/Liquibase instead, covered later in this course).

## 7. Best Practices

- Understand what SQL Hibernate is generating (`hibernate.show_sql=true` during development) — never treat it as a total black box.
- Use `hibernate.hbm2ddl.auto=validate` (not `update/create`) in production, relying on proper database migration tools for actual schema changes.

## 8. Production Usage

- Hibernate (typically via Spring Data JPA, covered in a later section, which builds ON TOP of Hibernate) is the dominant data-access approach in the vast majority of modern Java enterprise applications.

## 9. Frequently Asked Interview Questions

- 1 **What is ORM, and what problem does it solve?** A technique that automatically translates between Java objects and relational database rows/columns, eliminating the repetitive, error-prone manual mapping boilerplate required with raw JDBC.
  - 2 **What is the "Object-Relational Impedance Mismatch"?** The fundamental difference between how object-oriented languages (identity via reference, native inheritance) and relational databases (identity via primary key, no native inheritance) model data — the core theoretical reason ORM mapping is genuinely non-trivial.
  - 3 **What's the relationship between Hibernate and JPA?** JPA is a SPECIFICATION (interfaces/annotations); Hibernate is the most popular CONCRETE IMPLEMENTATION of that specification.
  - 4 **Does Hibernate replace JDBC?** No — Hibernate USES JDBC underneath; it generates and executes the SQL for you, rather than replacing the underlying mechanism.
  - 5 **Why is `hibernate.hbm2ddl.auto=update` risky in production?** It lets Hibernate automatically alter your database schema based on entity changes, which can be unpredictable/dangerous in production — proper schema migrations (Flyway/Liquibase) should be used instead.
- 
- 

## TOPIC 2: SESSIONFACTORY

---

### 1. Introduction

**What is it?** `SessionFactory` is a HEAVYWEIGHT, THREAD-SAFE object that represents Hibernate's connection to a SPECIFIC database — it's built ONCE, when your application starts, and reused for the ENTIRE application's lifetime to create individual `Session` objects (covered next) as needed.

**Why it's designed this way:** Building a `SessionFactory` involves expensive work — reading ALL your entity mappings, connecting to the database, building internal caches of metadata. Doing this ONCE at startup (not per-request) is a deliberate performance design, exactly analogous to why you use a CONNECTION POOL (JDBC section) instead of creating a raw connection per request.

## 2. Code Example — Building a SessionFactory

```
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public class HibernateUtil {
    private static final SessionFactory sessionFactory = buildSessionFactory();

    private static SessionFactory buildSessionFactory() {
        try {
            return new Configuration()
                .configure("hibernate.cfg.xml") // reads the config file shown in Topic 1
                .addAnnotatedClass(Employee.class) // register each @Entity class
                .addAnnotatedClass(Department.class)
                .buildSessionFactory();
        } catch (Throwable ex) {
            throw new ExceptionInInitializerError(ex);
        }
    }

    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }
}
```

**Critical rule:** Build EXACTLY ONE `SessionFactory` per database, ONCE, and reuse it for the application's entire lifetime — NEVER create a new one per request/operation (an extremely common, serious beginner performance mistake).

## 3. Real Life Analogy

- **Bank:** `SessionFactory` is like a bank BRANCH's entire physical infrastructure and staff setup — built ONCE (expensive to establish), then used repeatedly to handle MANY individual customer interactions (`Sessions`) throughout the day, without rebuilding the branch itself for every single customer.

## 4. Frequently Asked Interview Questions

- 1 **What is `SessionFactory`?** A heavyweight, thread-safe object representing Hibernate's connection to a database, used to create individual `Session` objects; built once per application lifetime.
- 2 **Why shouldn't you create a new `SessionFactory` for every operation?** Building it is expensive (reads all entity mappings, establishes database connectivity) — doing this repeatedly would severely hurt performance, exactly like creating a new raw JDBC connection per request instead of using a pool.
- 3 **Is `SessionFactory` thread-safe?** Yes — it's designed to be shared safely across your entire application's threads.

---

## TOPIC 3: SESSION

---

# 1. Introduction

**What is it?** A `Session` is a LIGHTWEIGHT, SHORT-LIVED, NOT-thread-safe object representing a SINGLE unit of work/conversation with the database — created FROM a `SessionFactory`, used for a specific operation or request, then closed. It's the primary interface for actually saving, loading, updating, and deleting entities.

## 2. `SessionFactory` vs `Session` — Comparison Table

| Aspect         | <code>SessionFactory</code>          | <code>Session</code>                                      |
|----------------|--------------------------------------|-----------------------------------------------------------|
| Lifetime       | ONE per application, entire lifetime | ONE per request/unit of work, SHORT-lived                 |
| Thread-safety  | Thread-safe, safely shared           | NOT thread-safe — one per thread/request                  |
| Cost to create | Expensive (created ONCE)             | Cheap (created MANY times)                                |
| Analogy        | The bank branch itself               | An individual customer service interaction at the counter |

## 3. Code Example — The Core CRUD Operations

```
import org.hibernate.Session;
import org.hibernate.Transaction;

public class EmployeeDao {

    public void saveEmployee(Employee emp) {
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            Transaction tx = session.beginTransaction();
            session.persist(emp); // generates and runs INSERT
            tx.commit();
        }
    }

    public Employee findEmployee(Long id) {
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            return session.get(Employee.class, id); // generates and runs SELECT by primary key
        }
    }

    public void updateSalary(Long id, Double newSalary) {
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            Transaction tx = session.beginTransaction();
            Employee emp = session.get(Employee.class, id);
            emp.setSalary(newSalary); // NO explicit "update" call needed here!
            tx.commit(); // Hibernate AUTOMATICALLY detects the change and runs UPDATE
        }
    }

    public void deleteEmployee(Long id) {
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            Transaction tx = session.beginTransaction();
            Employee emp = session.get(Employee.class, id);
            session.remove(emp); // generates and runs DELETE
            tx.commit();
        }
    }
}
```

## 4. "Dirty Checking" — Why `updateSalary()` Above Needed NO Explicit Update Call

This is one of Hibernate's most important, and most surprising-to-beginners, features: while an entity is being managed by an `ACTIVE Session` (within a transaction), Hibernate automatically tracks its `ORIGINAL` loaded state. When the transaction commits, Hibernate compares the `CURRENT` state against that original snapshot — if ANY field changed ("the entity is dirty"), Hibernate `AUTOMATICALLY` generates and runs the appropriate `UPDATE SQL`, with NO explicit "save"/"update" call required from you.

## 5. The Four Entity States (Object Lifecycle)

```
new Employee()  
|  
[TRANSIENT] <- just a plain Java object, Hibernate knows NOTHING about it  
|  
session.persist(emp)  
|  
[PERSISTENT] <- managed by an active Session, changes are AUTO-TRACKED (dirty checking)  
|  
session.close() (or the transaction ends)  
|  
[DETACHED] <- was persistent, but the Session is now closed; changes NO LONGER auto-tracked  
|  
session.remove(emp)  
|  
[REMOVED] <- marked for deletion, will be deleted from the DB on commit
```

## 6. Real Life Analogy

- **Restaurant:** A `Session` is like ONE specific waiter's interaction with ONE table during ONE visit — handling that table's specific orders from arrival to bill payment, then done. "Dirty checking" is like the waiter automatically noticing a customer changed their order mid-meal and updating the kitchen ticket accordingly, without the customer needing to explicitly say "please update my order" — the waiter is actively `WATCHING` (managing) that table for the whole visit.

## 7. Edge Cases

| Case                                                                                                 | Result                                                                                                                                               |
|------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Modifying a <code>DETACHED</code> entity's field (after <code>session.close()</code> )               | The change is NOT automatically tracked/saved — you'd need to explicitly <code>session.merge()</code> it back into a new active <code>Session</code> |
| Calling a lazy-loading getter on a <code>DETACHED</code> entity (see <code>Fetch Types</code> topic) | Throws <code>LazyInitializationException</code> — a very common real-world Hibernate error                                                           |
| Using ONE <code>Session</code> across MULTIPLE THREADS simultaneously                                | Undefined/dangerous behavior — <code>Session</code> is explicitly NOT thread-safe                                                                    |

## 8. Common Mistakes

- Sharing a single `Session` across multiple threads/requests — a serious, common bug, since `Session` is not thread-safe.

- Forgetting a `Session` needs to be `close()`d (or used in try-with-resources, as shown above) to release its underlying database connection back to the pool.
- Being surprised that modifying a persistent entity's field, THEN committing, automatically saves the change — this is intended "dirty checking" behavior, not a bug.

## 9. Frequently Asked Interview Questions

- 1 **What is a `Session` in Hibernate?** A lightweight, short-lived, NOT-thread-safe object representing a single unit of work with the database, used to perform CRUD operations on entities.
  - 2 **What are the four entity states in Hibernate's object lifecycle?** Transient, Persistent, Detached, Removed.
  - 3 **What is "dirty checking"?** Hibernate automatically detecting that a `PERSISTENT` entity's fields have changed (compared to when it was loaded) and generating the corresponding `UPDATE` SQL automatically at commit time, with no explicit "update" call needed.
  - 4 **What happens if you access a lazy-loaded association on a `DETACHED` entity?**  
`LazyInitializationException` — a very common real-world Hibernate error, since the `Session` needed to fetch that data on-demand is already closed.
  - 5 **Is `Session` thread-safe?** No — one `Session` should be used by exactly one thread for one unit of work, never shared concurrently.
- 

# TOPIC 4: ENTITY MAPPING

---

## 1. Introduction

**What is it?** Entity mapping is the process of annotating a plain Java class so Hibernate understands EXACTLY how it corresponds to a database table — which class maps to which table, which fields map to which columns, and what constraints apply.

## 2. Core Mapping Annotations — Full Reference

```
import jakarta.persistence.*;
import java.time.LocalDate;

@Entity // marks this class as a Hibernate-managed entity
@Table(name = "employees") // maps to the "employees" table (optional if names already match)
public class Employee {

    @Id // marks the PRIMARY KEY field
    @GeneratedValue(strategy = GenerationType.IDENTITY) // auto-increment, DB-generated (matches
    PostgreSQL's SERIAL)
    @Column(name = "emp_id")
    private Long id;

    @Column(name = "first_name", nullable = false, length = 50)
    private String firstName;

    @Column(name = "last_name", nullable = false, length = 50)
    private String lastName;

    @Column(precision = 10, scale = 2)
    private java.math.BigDecimal salary;

    @Column(name = "hire_date")
    private LocalDate hireDate;

    @Transient // this field is NEVER persisted to the database at all
    private String temporaryNote;

    @Enumerated(EnumType.STRING) // store an enum as its NAME (readable), not its ordinal (fragile!)
    private EmployeeStatus status;

    // no-argument constructor is REQUIRED by Hibernate (it instantiates entities via reflection)
    public Employee() { }

    // getters and setters...
}

enum EmployeeStatus { ACTIVE, ON_LEAVE, TERMINATED }
```

## 3. GenerationType Strategies

| Strategy | Behavior                                                                                 |
|----------|------------------------------------------------------------------------------------------|
| IDENTITY | Relies on the DATABASE's own auto-increment mechanism (e.g., PostgreSQL's SERIAL)        |
| SEQUENCE | Uses a database SEQUENCE object explicitly — often more efficient for batch inserts      |
| AUTO     | Lets Hibernate PICK the appropriate strategy based on the database dialect in use        |
| TABLE    | Uses a separate database table to simulate a sequence — rarely used, portable but slower |



## 4. `@Enumerated` — Why `EnumType.STRING` Is Almost Always Correct

```
@Enumerated(EnumType.ORDINAL) // BAD - stores 0, 1, 2... - if you ever REORDER the enum, existing data
silently
// means something DIFFERENT (see Core Java Topic 33's ordinal() warning!)

@Enumerated(EnumType.STRING) // GOOD - stores "ACTIVE", "ON_LEAVE" - readable, safe even if enum order
changes
```

## 5. Real Life Analogy

- **College:** Entity mapping is like filling out an official cross-reference form telling the college registrar EXACTLY which fields on a `Student` object (a Java class) correspond to which columns in the official student records database — "this Java field goes in THIS exact database column, with THESE exact rules."

## 6. Edge Cases

| Case                                                                                   | Result                                                                                                        |
|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| An <code>@Entity</code> class with NO no-argument constructor                          | Hibernate throws an error — it MUST be able to instantiate entities via reflection using a no-arg constructor |
| A field annotated <code>@Transient</code>                                              | NEVER read from or written to the database — purely an in-memory, non-persistent field                        |
| Using <code>EnumType.ORDINAL</code> and later reordering/inserting a new enum constant | Silently CORRUPTS existing stored data's meaning — a serious, easily-made real-world mistake                  |

## 7. Common Mistakes

- Forgetting the mandatory no-argument constructor on an `@Entity` class.
- Using `EnumType.ORDINAL` instead of `EnumType.STRING` — fragile against any future enum reordering.
- Not specifying `nullable = false/length` constraints on `@Column`, missing an opportunity to let Hibernate generate a more correct/constrained schema (when using `hbm2ddl.auto`).

## 8. Frequently Asked Interview Questions

- 1 **What does `@Entity` do?** Marks a class as a Hibernate-managed persistent entity, corresponding to a database table.
- 2 **What does `@Id` mark?** The field representing the entity's primary key.
- 3 **Why does every `@Entity` class need a no-argument constructor?** Because Hibernate instantiates entities via REFLECTION, which requires a no-arg constructor to create the object before populating its fields.
- 4 **Why is `EnumType.STRING` generally preferred over `EnumType.ORDINAL`?** `ORDINAL` stores the enum's numeric position, which silently changes meaning if the enum's constants are ever reordered or new ones inserted in the middle — `STRING` stores the readable name, immune to this fragility.
- 5 **What does `@Transient` do?** Marks a field that Hibernate should completely ignore — it's never read from or written to the database.

---

# TOPIC 5: RELATIONSHIPS

---

## 1. Introduction

**What is it?** Relationship mapping lets Hibernate understand and manage FOREIGN-KEY-based associations between entities (exactly the relational concepts from the SQL section's Primary & Foreign Keys / Joins topics), expressed naturally as Java OBJECT REFERENCES instead of raw foreign key values — e.g., `employee.getDepartment()` returns an ACTUAL `Department` object, not just a numeric `dept_id`.

## 2. @ManyToOne — Many Employees, One Department

```
@Entity
public class Employee {
    @Id @GeneratedValue private Long id;
    private String firstName;

    @ManyToOne // MANY employees belong to ONE department
    @JoinColumn(name = "dept_id") // the actual foreign key column name in the "employees" table
    private Department department;
}
```

## 3. @OneToMany — One Department, Many Employees (The Inverse Side)

```
@Entity
public class Department {
    @Id @GeneratedValue private Long id;
    private String deptName;

    @OneToMany(mappedBy = "department") // "department" = the FIELD NAME on the Employee side (Topic 2 above)
    private List<Employee> employees = new ArrayList<>();
}
```

`mappedBy` tells Hibernate: "the FOREIGN KEY for this relationship is actually owned/managed by the OTHER side (`Employee.department`) — I'm just the INVERSE, read-only-in-terms-of-the-FK-column view of that same relationship." This is a CRITICAL concept: **exactly ONE side of a bidirectional relationship "owns" the foreign key column; the other side is purely for convenient object-graph navigation.**

## 4. @OneToOne — One Employee, One Parking Spot

```
@Entity
public class Employee {
    @Id @GeneratedValue private Long id;

    @OneToOne
    @JoinColumn(name = "parking_spot_id")
    private ParkingSpot parkingSpot;
}

@Entity
public class ParkingSpot {
    @Id @GeneratedValue private Long id;
    private String spotNumber;

    @OneToOne(mappedBy = "parkingSpot")
    private Employee employee;
}
```

## 5. @ManyToMany — Employees and Projects (Many-to-Many Needs a Join Table)

```
@Entity
public class Employee {
    @Id @GeneratedValue private Long id;

    @ManyToMany
    @JoinTable(
        name = "employee_project", // the actual JOIN TABLE's name (like a pure SQL many-to-many)
        joinColumns = @JoinColumn(name = "emp_id"),
        inverseJoinColumns = @JoinColumn(name = "project_id")
    )
    private Set<Project> projects = new HashSet<>();
}

@Entity
public class Project {
    @Id @GeneratedValue private Long id;
    private String projectName;

    @ManyToMany(mappedBy = "projects")
    private Set<Employee> employees = new HashSet<>();
}
```

## 6. Full CRUD Example — Saving Related Entities

```
try (Session session = HibernateUtil.getSessionFactory().openSession()) {
    Transaction tx = session.beginTransaction();

    Department engineering = new Department();
    engineering.setDeptName("Engineering");

    Employee alice = new Employee();
    alice.setFirstName("Alice");
    alice.setDepartment(engineering); // set the OWNING side (Employee -> Department)

    session.persist(engineering);
    session.persist(alice);

    tx.commit();
}

// Later, LOADING an employee automatically gives you REAL object navigation:
try (Session session = HibernateUtil.getSessionFactory().openSession()) {
    Employee emp = session.get(Employee.class, aliceId);
    System.out.println(emp.getDepartment().getDeptName()); // "Engineering" - navigated as a real object,
    NOT a raw ID!
}
```

## 7. Real Life Analogy

- **College:** `@ManyToOne` (Employee → Department) is like every student's record having a direct link to THEIR ONE department. `@OneToMany` (Department → Employees) is like a department's record being able to list ALL its enrolled students — the SAME underlying relationship, viewed from two different directions, but the actual foreign key "belongs" conceptually to the student side (`mappedBy` points there). `@ManyToMany` (Employee ↔ Projects) is like students who can enroll in MULTIPLE clubs, and clubs having MULTIPLE members — needing a separate join table to represent all the combinations.

## 8. Edge Cases

| Case                                                                                           | Result                                                                                                                                                                            |
|------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Setting the relationship ONLY on the <code>mappedBy</code> (inverse) side, not the owning side | The foreign key is NOT actually updated in the database — a very common, confusing beginner bug; you MUST set the relationship on the OWNING side for it to be persisted          |
| A bidirectional relationship missing <code>mappedBy</code> on one side                         | Hibernate may create an UNEXPECTED extra join table, thinking these are two SEPARATE, unrelated relationships instead of one bidirectional one                                    |
| Circular <code>toString()</code> methods on both sides of a bidirectional relationship         | <code>StackOverflowError</code> — <code>Employee.toString()</code> calls <code>Department.toString()</code> which calls back into <code>Employee.toString()</code> ... infinitely |

## 9. Common Mistakes

- Setting a relationship ONLY on the inverse (`mappedBy`) side and being confused why the database's foreign key column wasn't updated — always set it on the OWNING side.

- Forgetting `mappedBy`, accidentally creating TWO separate, redundant relationships/join tables instead of one properly bidirectional one.
- Writing `toString()/equals()/hashCode()` that recursively include BOTH sides of a bidirectional relationship, causing infinite recursion.

## 10. Best Practices

- Always update BOTH sides of a bidirectional relationship together (often via a helper method, e.g., `department.addEmployee(employee)` that sets both `employee.setDepartment(department)` AND adds to `department.getEmployees()`, to keep the in-memory object graph consistent.
- Be deliberate about which side "owns" the relationship (has the actual `@JoinColumn`) — typically the "many" side in a one-to-many, matching where the foreign key column naturally lives in the database.
- Avoid including bidirectional relationship fields in `toString()/equals()/hashCode()` to prevent infinite recursion.

## 11. Frequently Asked Interview Questions

- 1 **What does `mappedBy` indicate?** That THIS side of the relationship is the INVERSE (non-owning) side — the actual foreign key column is managed by the field named in `mappedBy`, on the other entity.
- 2 **What happens if you only set a relationship on the inverse (`mappedBy`) side?** The foreign key in the database is NOT updated — you must set the relationship on the OWNING side for the change to actually persist.
- 3 **Why does `@ManyToMany` require a separate join table?** Because a many-to-many relationship cannot be represented by a single foreign key column on either side — it fundamentally requires an intermediate table listing all valid combinations, exactly like plain relational SQL design.
- 4 **What's a common bug with bidirectional relationships and `toString()`?** Infinite recursion (`StackOverflowError`) if both sides' `toString()` methods include the other side, each triggering the other endlessly.

---

# TOPIC 6: FETCH TYPES

---

## 1. Introduction

**What is it?** Fetch type determines WHEN Hibernate actually loads a related entity/collection from the database — **EAGER** (immediately, as part of loading the parent) or **LAZY** (deferred, only loaded the FIRST time it's actually accessed in code).

## 2. Code Example — Specifying Fetch Type

```
@Entity
public class Employee {
    @ManyToOne(fetch = FetchType.EAGER) // Department is loaded IMMEDIATELY alongside the Employee
    private Department department;

    @OneToMany(mappedBy = "department", fetch = FetchType.LAZY) // employees are loaded ONLY when actually
    accessed
    private List<Employee> employees;
}
```

## 3. Default Fetch Types (An Important, Frequently-Tested Detail)

| Relationship | Default Fetch Type        |
|--------------|---------------------------|
| @ManyToOne   | <b>EAGER</b> (by default) |
| @OneToOne    | <b>EAGER</b> (by default) |
| @OneToMany   | <b>LAZY</b> (by default)  |
| @ManyToMany  | <b>LAZY</b> (by default)  |

## 4. The Trade-off — Why LAZY Isn't Always "Better"

**EAGER:** loads the Department THE MOMENT the Employee is loaded, even if you never actually use it  
-> WASTES a query/join if the data isn't needed - but data is ALWAYS ready when accessed, safely

**LAZY:** loads the Department ONLY when employee.getDepartment().getDeptName() is ACTUALLY called  
-> SAVES unnecessary loading when not needed - but REQUIRES an active Session at that exact moment, or throws LazyInitializationException (see Topic 3 and the dedicated Lazy vs Eager topic)

## 5. Real Life Analogy

- **Restaurant:** EAGER loading is like a waiter bringing the ENTIRE meal (starter, main, dessert) to the table all at once, whether or not the customer wants dessert right away. LAZY loading is like bringing dishes ONE AT A TIME, only when the customer actually asks for the next course — more efficient IF they don't want everything, but requires the kitchen (Session) to still be "open" when they DO ask.

## 6. Frequently Asked Interview Questions

- 1 **What's the default fetch type for @ManyToOne and @OneToOne?** EAGER.
- 2 **What's the default fetch type for @OneToMany and @ManyToMany?** LAZY.
- 3 **What's the risk of using EAGER fetching everywhere?** It can load FAR more data than actually needed for a given operation, hurting performance — potentially cascading into loading large, deeply nested object graphs unnecessarily.
- 4 **What's the risk of LAZY fetching?** Accessing the lazy data AFTER the Session has closed throws `LazyInitializationException` — a very common real-world Hibernate error (full detail in the dedicated Lazy vs Eager Loading topic).

# TOPIC 7: CASCADE TYPES

## 1. Introduction

**What is it?** Cascading determines whether an operation performed on a PARENT entity (save, update, delete) should AUTOMATICALLY be PROPAGATED to its related CHILD entities — e.g., should deleting a Department automatically delete all its Employees too?

## 2. Code Example

```
@Entity
public class Department {
    @Id @GeneratedValue private Long id;

    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Employee> employees = new ArrayList<>();
}

Department dept = new Department();
Employee emp1 = new Employee("Alice");
Employee emp2 = new Employee("Bob");
dept.getEmployees().add(emp1);
dept.getEmployees().add(emp2);

session.persist(dept);
// Because of CascadeType.ALL (or specifically CascadeType.PERSIST), saving 'dept' ALSO
// automatically saves emp1 and emp2 - NO separate session.persist(emp1) call needed!
```

## 3. The Cascade Types — Full Reference

| Cascade Type | Effect                                                    |
|--------------|-----------------------------------------------------------|
| PERSIST      | Saving the parent ALSO saves new children                 |
| MERGE        | Merging (updating) the parent ALSO merges children        |
| REMOVE       | Deleting the parent ALSO deletes children                 |
| REFRESH      | Refreshing the parent from the DB ALSO refreshes children |
| DETACH       | Detaching the parent ALSO detaches children               |
| ALL          | ALL of the above, combined                                |

## 4. `orphanRemoval` — A Related but DISTINCT Concept

```
dept.getEmployees().remove(emp1); // simply REMOVING emp1 from the LIST (not deleting it via
session.remove())
// With orphanRemoval = true: emp1 is AUTOMATICALLY DELETED from the database too,
// since it's no longer referenced by ANY parent - it became an "orphan"
// WITHOUT orphanRemoval: emp1 simply has its dept_id set to NULL (or the relationship
// is otherwise just disconnected), but the employee row itself still exists
```

## 5. Real Life Analogy

- **Bank:** `CascadeType.PERSIST` is like closing a "master" business account AUTOMATICALLY creating and setting up all its associated linked sub-accounts in one bundled action, rather than needing to manually open each one separately. `orphanRemoval` is like automatically closing a sub-account entirely the moment it's disconnected from EVERY master account, rather than leaving it dangling, ownerless, in the system.

## 6. Edge Cases

| Case                                                                                                                  | Result                                                                                                                                                                                                                                                             |
|-----------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>CascadeType.REMOVE</code> on a <code>@ManyToOne</code> relationship (many employees pointing to one department) | DANGEROUS if misapplied — deleting ONE employee could unintentionally attempt to cascade-delete the ENTIRE shared department (affecting other employees too); cascade REMOVE generally makes sense on the "one" side owning/containing the "many," NOT the reverse |
| No cascade specified at all                                                                                           | You must EXPLICITLY save/delete each related entity yourself, separately                                                                                                                                                                                           |

## 7. Common Mistakes

- Applying `CascadeType.REMOVE` (or ALL) carelessly on relationships where it doesn't make logical sense, accidentally deleting shared/unrelated data.
- Confusing `orphanRemoval` with cascade REMOVE — they solve related but DIFFERENT problems (`orphanRemoval` reacts to DISCONNECTION from the collection; cascade REMOVE reacts to explicit deletion of the parent).

## 8. Frequently Asked Interview Questions

- 1 **What does cascading control in Hibernate?** Whether an operation (save/update/delete) on a parent entity automatically propagates to its related child entities.
- 2 **What does `CascadeType.ALL` include?** `PERSIST`, `MERGE`, `REMOVE`, `REFRESH`, `DETACH` — all cascade behaviors combined.
- 3 **What's the difference between `orphanRemoval` and `CascadeType.REMOVE`?** `orphanRemoval` deletes a child when it's simply REMOVED from its parent's collection (disconnected); `CascadeType.REMOVE` deletes children when the PARENT ITSELF is explicitly deleted — related but distinct triggers.
- 4 **When would applying `CascadeType.REMOVE` be dangerous?** On a relationship where the "child" is actually SHARED across multiple parents — cascading delete could unintentionally remove data still needed by other parents.

# TOPIC 8: HQL (HIBERNATE QUERY LANGUAGE)



## 1. Introduction

**What is it?** HQL is an object-oriented query language that looks a lot like SQL, but queries against your JAVA ENTITY CLASSES and their FIELDS, rather than raw database tables and columns — Hibernate translates HQL into the actual underlying SQL for your specific database automatically.

## 2. HQL vs SQL — The Key Difference

```
-- Raw SQL (queries the actual TABLE and COLUMN names)
SELECT * FROM employees WHERE salary > 50000;

// HQL (queries the ENTITY CLASS and FIELD names - "Employee" and "salary", not "employees"/"salary"
table/column)
String hql = "FROM Employee e WHERE e.salary > 50000";
List<Employee> results = session.createQuery(hql, Employee.class).getResultList();
```

## 3. Code Examples — Common HQL Patterns

```
// Select specific fields (not whole entities)
List<String> names = session.createQuery(
"SELECT e.firstName FROM Employee e WHERE e.dept.deptName = :deptName", String.class)
.setParameter("deptName", "Engineering")
.getResultList();

// Named parameters (ALWAYS prefer these over string concatenation - same SQL Injection lesson as
JDBC!)
List<Employee> highEarnings = session.createQuery(
"FROM Employee e WHERE e.salary > :minSalary", Employee.class)
.setParameter("minSalary", 80000)
.getResultList();

// JOIN across the object relationship (navigates the Java field, NOT a raw foreign key column)
List<Employee> engineers = session.createQuery(
"SELECT e FROM Employee e JOIN e.department d WHERE d.deptName = :name", Employee.class)
.setParameter("name", "Engineering")
.getResultList();

// Aggregate functions
Long count = session.createQuery(
"SELECT COUNT(e) FROM Employee e WHERE e.dept.deptName = :name", Long.class)
.setParameter("name", "Engineering")
.getSingleResult();

// UPDATE via HQL (bulk operation, bypasses individual entity dirty-checking for efficiency)
int rowsUpdated = session.createMutationQuery(
"UPDATE Employee e SET e.salary = e.salary * 1.1 WHERE e.dept.deptName = :name")
.setParameter("name", "Engineering")
.executeUpdate();
```

## 4. Real Life Analogy

- **Library:** SQL is like asking the library's PHYSICAL SHELF SYSTEM directly ("go to Shelf B, Row 3"). HQL is like asking a librarian in terms of the BOOK'S actual SUBJECT/TITLE ("find me science fiction novels by this author") — the librarian (Hibernate) then translates that into the exact physical shelf lookups needed, without you needing to know the raw physical layout.

## 5. Edge Cases

| Case                                                                                                                            | Result                                                                                                                                     |
|---------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Using the wrong CASE for an entity/field name (HQL is CASE-SENSITIVE for these, unlike table/column names in many SQL dialects) | Query fails — HQL entity/field names must exactly match your Java class/field names                                                        |
| String-concatenating user input directly into HQL instead of using named parameters                                             | Still vulnerable to injection-style attacks — ALWAYS use <code>setParameter()</code> , exactly like <code>PreparedStatement</code> in JDBC |

## 6. Common Mistakes

- Concatenating user input directly into an HQL string instead of using named parameters — the SAME security risk as raw SQL's `Statement` vs `PreparedStatement` issue (JDBC section).
- Confusing HQL's entity/field names with the underlying table/column names — they can DIFFER if you've used `@Table/@Column` name overrides.

## 7. Frequently Asked Interview Questions

- 1 **What is HQL?** Hibernate Query Language — an object-oriented query language resembling SQL, but querying entity CLASSES and FIELDS rather than raw tables and columns.
- 2 **What's the key difference between HQL and SQL?** HQL operates on your JAVA ENTITY model (class/field names); SQL operates directly on the database's actual table/column names — Hibernate translates HQL into the correct underlying SQL.
- 3 **Should you ever concatenate user input directly into an HQL string?** No — exactly like raw JDBC's `Statement` vs `PreparedStatement`, always use named parameters (`setParameter()`) to avoid injection-style vulnerabilities.

---

# TOPIC 9: CRITERIA API

---

## 1. Introduction

**What is it?** The Criteria API lets you build queries PROGRAMMATICALLY using Java method calls, rather than writing a query as a STRING (like HQL/SQL) — enabling TYPE-SAFE, dynamically-constructed queries that the COMPILER can check, and that are easier to build up conditionally based on runtime logic.

## 2. Why It Exists — The Problem With String-Based Queries

```
// HQL as a STRING - a typo like "salery" instead of "salary" is only caught at RUNTIME
String hql = "FROM Employee e WHERE e.salery > :min"; // TYPO - compiles fine, fails at runtime!
```

```
// Criteria API - the EQUIVALENT field reference is CHECKED BY THE COMPILER
CriteriaBuilder cb = session.getCriteriaBuilder();
CriteriaQuery<Employee> query = cb.createQuery(Employee.class);
Root<Employee> root = query.from(Employee.class);
query.select(root).where(cb.gt(root.get("salary"), 50000)); // "salary" - a typo here is ALSO just a
runtime
// string lookup with metamodel-less Criteria,
// BUT the Static Metamodel (below) fixes this fully
```

### 3. Code Example — Building a Dynamic Query

```
CriteriaBuilder cb = session.getCriteriaBuilder();
CriteriaQuery<Employee> cq = cb.createQuery(Employee.class);
Root<Employee> employee = cq.from(Employee.class);

List<Predicate> predicates = new ArrayList<>();

if (minSalary != null) {
    predicates.add(cb.gt(employee.get("salary"), minSalary));
}
if (deptName != null) {
    Join<Employee, Department> dept = employee.join("department");
    predicates.add(cb.equal(dept.get("deptName"), deptName));
}

cq.select(employee).where(predicates.toArray(new Predicate[0]));
List<Employee> results = session.createQuery(cq).getResultList();
```

This DYNAMIC query-building pattern — adding conditions ONLY when relevant filter values are actually provided — is MUCH cleaner with the Criteria API than trying to conditionally build up an HQL/SQL STRING with careful string concatenation.

### 4. HQL vs Criteria API — When to Use Which

| Aspect              | HQL                                               | Criteria API                                                              |
|---------------------|---------------------------------------------------|---------------------------------------------------------------------------|
| Query form          | A STRING (parsed at runtime)                      | Built via Java METHOD CALLS                                               |
| Compile-time safety | None (typos only caught at runtime)               | Better, ESPECIALLY with a generated Static Metamodel                      |
| Best for            | Simple, fixed, well-known queries — more readable | Complex, DYNAMICALLY built queries with many optional/conditional filters |

### 5. Frequently Asked Interview Questions

- 1 What is the Criteria API?** A way to build Hibernate/JPA queries programmatically using Java method calls, rather than as a query STRING.
- 2 What problem does the Criteria API solve better than HQL?** Building DYNAMIC queries with many optional/conditional filter criteria — much cleaner than conditionally concatenating an HQL string.
- 3 What's a downside of HQL compared to the Criteria API?** HQL is a plain string, so typos in entity/field names are only caught at RUNTIME, not compile time.

# TOPIC 10: CACHING

---

## 1. Introduction

**What is it?** Hibernate provides MULTIPLE LEVELS of caching to avoid re-querying the database for data that's already been recently loaded — dramatically improving performance for repeated access to the same data.

## 2. First-Level Cache (L1) — Automatic, Per-Session, Always On

```
try (Session session = sessionFactory.openSession()) {
    Employee emp1 = session.get(Employee.class, 1L); // hits the DATABASE (first time)
    Employee emp2 = session.get(Employee.class, 1L); // returns the SAME cached object, NO second database hit!

    System.out.println(emp1 == emp2); // true - literally the SAME Java object reference
}
```

The First-Level Cache is tied to a SINGLE `Session` — it's ALWAYS enabled (cannot be disabled), and is cleared entirely once that `Session` is closed.

## 3. Second-Level Cache (L2) — Optional, Shared ACROSS Sessions

```
<!-- hibernate.cfg.xml -->
<property name="hibernate.cache.use_second_level_cache">true</property>
<property name="hibernate.cache.region.factory_class">org.hibernate.cache.jcache.internal.JCacheRegionFactory</property>

@Entity
@Cacheable
@org.hibernate.annotations.Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
public class Department {
    // ...
}
```

Unlike L1, the Second-Level Cache is SHARED across MULTIPLE `Sessions` (and potentially across the entire application), requiring an explicit caching provider (like Ehcache or Caffeine) to be configured — genuinely useful for FREQUENTLY-READ, RARELY-CHANGED data (like a list of departments or product categories), but adds real complexity around cache invalidation/consistency.

## 4. Real Life Analogy

- **Library:** The First-Level Cache is like a reader keeping a book they just retrieved on THEIR OWN desk for the duration of their CURRENT visit — asking for the SAME book again during that same visit doesn't require walking back to the shelf. The Second-Level Cache is like the library maintaining a SHARED "recently popular books" quick-access shelf near the entrance, available to ANY reader, across ANY visit, until that shelf's contents are refreshed/invalidated.

## 5. Edge Cases

| Case                                                                                             | Result                                                                                                                                  |
|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Data changed DIRECTLY in the database (bypassing Hibernate entirely, e.g., via a raw SQL script) | The Second-Level Cache becomes STALE, unaware of the external change, until explicitly invalidated/expired                              |
| Relying on L1 cache across TWO SEPARATE Sessions                                                 | Does NOT work — L1 cache is scoped to ONE session only; you'd need L2 (or simply accept a fresh database query) for cross-session reuse |

## 6. Common Mistakes

- Enabling Second-Level Caching broadly, by default, for frequently-CHANGING data — leads to serious stale-data bugs; L2 caching is best reserved for genuinely slow-changing, read-heavy reference data.
- Assuming the First-Level Cache helps across different Sessions — it doesn't; it's strictly per-Session.

## 7. Frequently Asked Interview Questions

- 1 **What is the First-Level Cache, and can it be disabled?** A per-Session, ALWAYS-ON cache — cannot be disabled; it avoids redundant database hits for the SAME entity within a single Session's lifetime.
- 2 **What is the Second-Level Cache, and how does it differ from the First-Level Cache?** An OPTIONAL cache SHARED across multiple Sessions (and potentially the whole application), requiring explicit configuration and a caching provider — unlike L1, which is per-Session and automatic.
- 3 **What kind of data is a good candidate for Second-Level Caching?** Frequently-READ, RARELY-CHANGED reference data (e.g., a list of departments/categories) — NOT frequently-changing transactional data.
- 4 **What risk does Second-Level Caching introduce?** Stale data — if the underlying database changes through a path that bypasses Hibernate's cache-awareness, the cache can serve outdated information until invalidated.

---

# TOPIC 11: LAZY VS EAGER LOADING (DEEP DIVE)

---

## 1. Introduction

*(Fetch types and their DEFAULT values were introduced in Topic 6. This topic is a focused, interview-critical deep dive into the SPECIFIC runtime behaviors, dangers, and fixes surrounding lazy loading — one of the single most commonly asked Hibernate interview topics.)*

## 2. `LazyInitializationException` — The #1 Real-World Hibernate Error

```
public Employee getEmployee(Long id) {
    try (Session session = sessionFactory.openSession()) {
        return session.get(Employee.class, id);
    } // <-- Session CLOSES here, right as the method returns
}

public void printDepartment(Long empId) {
    Employee emp = getEmployee(empId);
    System.out.println(emp.getDepartment().getDeptName()); // BOOM! LazyInitializationException!
    // department was LAZY-fetched, and by the time we try to access it, the Session that
    // would have loaded it on-demand is ALREADY CLOSED - Hibernate cannot fetch the data anymore
}
```

## 3. Why This Happens — The Proxy Mechanism

When you **LAZY**-load a relationship, Hibernate doesn't actually give you the **REAL** related object immediately — it gives you a lightweight **PROXY** object (a dynamically-generated subclass) that looks and behaves like the real thing, but contains **NO** actual data yet. The **FIRST** time you call a method on it (like `getDeptName()`), the proxy triggers Hibernate to run the **ACTUAL** database query, using the **ORIGINAL Session** — but **ONLY** if that Session is **STILL** open at that exact moment.

```
session.get(Employee.class, id)
|
v
Employee object returned, with `department` field = a PROXY object (no real data loaded yet)
|
v (Session CLOSES here)
|
emp.getDepartment().getDeptName()
|
v
Proxy tries to fetch real data -> needs the ORIGINAL Session -> Session is CLOSED -> EXCEPTION!
```

## 4. The Three Standard Fixes

**Fix 1: Fetch it EAGERLY (simplest, but loses lazy loading's benefits for that field)**

```
@ManyToOne(fetch = FetchType.EAGER)
private Department department;
```

**Fix 2: Access it WHILE the Session is still open (the recommended, most flexible fix)**

```
public Employee getEmployeeWithDept(Long id) {
    try (Session session = sessionFactory.openSession()) {
        Employee emp = session.get(Employee.class, id);
        emp.getDepartment().getDeptName(); // FORCE initialization WHILE the session is open
        return emp;
    }
}
```

**Fix 3: Use `JOIN FETCH` in HQL/JPQL to load everything in ONE query upfront**

```
String hql = "FROM Employee e JOIN FETCH e.department WHERE e.id = :id";
Employee emp = session.createQuery(hql, Employee.class)
    .setParameter("id", id)
    .getSingleResult();
// department is now FULLY loaded, in the SAME query, no separate lazy-load trip needed at all
```

## 5. The "N+1 Select Problem" — A Related, Critical Performance Issue

```
List<Employee> employees = session.createQuery("FROM Employee", Employee.class).getResultList();
// This runs 1 QUERY to get all employees...

for (Employee emp : employees) {
    System.out.println(emp.getDepartment().getDeptName()); // ...then triggers ONE SEPARATE query
} // PER EMPLOYEE to lazy-load their department!

// If there are 100 employees: 1 query for employees + 100 SEPARATE queries for departments = 101
TOTAL queries!
// This is the notorious "N+1 SELECT PROBLEM" - a genuinely serious, extremely common real-world
// Hibernate performance bug.
```

**The fix:** use `JOIN FETCH` (as shown above) to load employees AND their departments in a SINGLE query, instead of triggering N additional lazy-load queries in a loop.

```
String hql = "SELECT e FROM Employee e JOIN FETCH e.department"; // ONE query total, not 1+N!
List<Employee> employees = session.createQuery(hql, Employee.class).getResultList();
```

## 6. Real Life Analogy

- **Restaurant:** Lazy loading is like a waiter who only fetches a specific ingredient's PRICE from the supplier the MOMENT a customer actually asks about it — efficient, UNLESS the kitchen has already closed for the day (Session closed) by the time that question comes up, in which case the waiter simply CANNOT get the answer anymore. The N+1 problem is like a waiter walking back to the supplier SEPARATELY for EVERY SINGLE dish's ingredient price, one trip at a time, instead of asking for ALL the needed prices together in ONE trip upfront (`JOIN FETCH`).

## 7. Edge Cases

| Case                                                                                                                     | Result                                                                                                                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Accessing a lazy association within an ACTIVE, open transaction/Session                                                  | Works correctly — Hibernate transparently fetches the data on-demand                                                                                                                                                    |
| Serializing a lazy-loaded entity (e.g., to JSON for a REST API response) AFTER the Session has closed                    | Can throw <code>LazyInitializationException</code> (or produce broken/incomplete JSON) if the serializer tries to access uninitialized lazy fields — a VERY common real-world bug in Spring Boot REST APIs specifically |
| Using <code>JOIN FETCH</code> on a <code>@OneToMany</code> collection alongside pagination ( <code>LIMIT/OFFSET</code> ) | Can produce WARNING/incorrect results in some Hibernate versions — a known, documented limitation requiring careful handling (e.g., separate count queries)                                                             |

## 8. Common Mistakes

- Designing REST API layers that return `@Entity` objects DIRECTLY (with lazy relationships) — the JSON serialization often happens OUTSIDE the Hibernate Session's scope, causing `LazyInitializationException` in production; the fix is to map entities to separate DTOs WHILE the Session is still open (a strong argument for the DTO pattern used throughout later Spring sections).
- Writing loops that lazily access a relationship per-iteration, unknowingly causing the N+1 problem.

- Overusing EAGER fetching everywhere "just to avoid the exception" — trades one problem (occasional exceptions) for a worse one (unnecessarily loading large amounts of unneeded data on EVERY query, everywhere).

## 9. Best Practices

- Prefer LAZY as the default mindset, and use JOIN FETCH (or DTO projections) DELIBERATELY when you know upfront you'll need related data — rather than defaulting to EAGER everywhere.
- Map entities to DTOs WHILE the Session is open, before returning data across application layer boundaries (especially before JSON serialization in a REST API).
- Watch for the N+1 problem specifically whenever a loop accesses a lazy relationship — a huge, extremely common real-world performance bug in production Hibernate/JPA applications.

## 10. Production Usage

- The N+1 problem and LazyInitializationException are, by a wide margin, among the MOST commonly encountered real-world Hibernate bugs in production Spring Boot applications — understanding this topic deeply is directly, immediately practically useful for real job debugging, not just interviews.

## 11. Frequently Asked Interview Questions

- 1 **What causes LazyInitializationException?** Accessing a lazily-loaded relationship AFTER its originating Session has already been closed — the proxy object needs an active Session to fetch its real data on first access, and none is available.
- 2 **What are the three standard fixes for LazyInitializationException?** Fetch eagerly instead, access the lazy field WHILE the Session is still open, or use JOIN FETCH to load everything in one upfront query.
- 3 **What is the "N+1 Select Problem"?** A performance bug where fetching N parent entities, then lazily accessing a related field for EACH one in a loop, triggers 1 initial query PLUS N additional individual queries (N+1 total), instead of one single, efficient combined query.
- 4 **How do you fix the N+1 problem?** Use JOIN FETCH in HQL/JPQL (or the equivalent in Spring Data JPA, covered later) to load the parent AND its related data together in ONE query.
- 5 **Why is directly returning @Entity objects from a REST API controller risky?** JSON serialization can happen AFTER the Hibernate Session has closed, triggering LazyInitializationException on any un-initialized lazy field — mapping to a separate DTO while the Session is still open avoids this entirely.



## 12. Revision Notes (Entire Hibernate Section Consolidated)

### HIBERNATE – CHEAT SHEET

=====

ORM SOLVES: repetitive JDBC boilerplate + the Object-Relational Impedance Mismatch  
JPA = specification (interfaces/annotations); HIBERNATE = the implementation

SessionFactory: ONE per app, expensive, thread-safe, built ONCE at startup  
Session: per-request/unit-of-work, cheap, NOT thread-safe, use try-with-resources

ENTITY STATES: Transient -> Persistent (auto dirty-checked!) -> Detached -> Removed

MAPPING: @Entity @Id @GeneratedValue @Column @Transient @Enumerated(String not ORDINAL!)  
requires a no-arg constructor (Hibernate uses reflection)

RELATIONSHIPS: @ManyToOne (default EAGER) @OneToMany (default LAZY, mappedBy on inverse side) @OneToOne @ManyToMany (needs @JoinTable)  
SET the relationship on the OWNING side, or the FK never actually updates!

CASCADE: PERSIST/MERGE/REMOVE/REFRESH/DETACH/ALL - propagates parent operations to children  
orphanRemoval: deletes a child when DISCONNECTED from its parent's collection

HQL: query ENTITY/FIELD names (not table/column names), ALWAYS use named parameters  
CRITERIA API: type-safe, programmatic query building - best for DYNAMIC/conditional queries

CACHING: L1 (per-Session, ALWAYS on, automatic) vs L2 (shared, OPTIONAL, needs a cache provider, risk of staleness)

LAZY VS EAGER (the #1 real-world Hibernate problem):

LazyInitializationException -> accessing lazy data AFTER Session closed

FIX: fetch eagerly / access while Session open / use JOIN FETCH

N+1 PROBLEM -> loop lazily accessing a relation = 1+N queries instead of 1

FIX: JOIN FETCH, or map to DTOs while the Session is still open

## 13. Homework (Covers Entire Hibernate Section)

- 1 Set up Hibernate against your `company_db` PostgreSQL database (from the PostgreSQL section), define `Employee/Department` entities with a bidirectional `@ManyToOne/@OneToMany`, and perform full CRUD operations.
- 2 Deliberately trigger a `LazyInitializationException` (return an entity from a method after closing its Session, then access a lazy field), then fix it using `JOIN FETCH`.
- 3 Deliberately trigger the N+1 problem (loop over a list of parents, lazily accessing each one's children), observe the extra queries via `hibernate.show_sql=true`, then fix it with `JOIN FETCH` and confirm the query count drops to one.
- 4 Write an HQL query using `JOIN`, an aggregate function, and a named parameter, comparing it against the equivalent raw SQL from the SQL section.

## 14. Mini Project — A Simple Employee Management DAO Layer

```
package com.jobsradar.hibernate;

import org.hibernate.Session;
import org.hibernate.Transaction;
import java.util.List;

public class EmployeeService {

    public Employee create(String firstName, Double salary, Long deptId) {
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            Transaction tx = session.beginTransaction();
            Department dept = session.get(Department.class, deptId);
            Employee emp = new Employee();
            emp.setFirstName(firstName);
            emp.setSalary(salary);
            emp.setDepartment(dept);
            session.persist(emp);
            tx.commit();
            return emp;
        }
    }

    public List<Employee> findByDepartmentWithFetch(String deptName) {
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            String hql = "SELECT e FROM Employee e JOIN FETCH e.department d WHERE d.deptName = :name";
            return session.createQuery(hql, Employee.class)
                .setParameter("name", deptName)
                .getResultList();
        }
    }

    public void giveRaise(Long empId, double percent) {
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            Transaction tx = session.beginTransaction();
            Employee emp = session.get(Employee.class, empId);
            emp.setSalary(emp.getSalary() * (1 + percent / 100)); // dirty-checking handles the UPDATE automatically
            tx.commit();
        }
    }
}
```

---

## FINAL SUMMARY — HIBERNATE COMPLETE

This concludes the **Hibernate** section — 11 topics:

- 1 **ORM** — the exact JDBC boilerplate problem it eliminates, the Impedance Mismatch, Hibernate vs JPA.
- 2 **SessionFactory** — one per application, expensive, thread-safe.
- 3 **Session** — per-request, cheap, the four entity states, dirty checking.
- 4 **Entity Mapping** — `@Entity`/`@Id`/`@Column`/`@Enumerated`, the no-arg constructor requirement.
- 5 **Relationships** — `@ManyToOne`/`@OneToMany`/`@OneToOne`/`@ManyToMany`, owning vs inverse sides.
- 6 **Fetch Types** — `LAZY` vs `EAGER`, and each relationship's default.

- 7 **Cascade Types** — propagating operations to children, `orphanRemoval`.
- 8 **HQL** — object-oriented querying, named parameters.
- 9 **Criteria API** — type-safe, dynamic query building.
- 10 **Caching** — L1 (automatic, per-Session) vs L2 (optional, shared, riskier).
- 11 **Lazy vs Eager Loading (deep dive)** — `LazyInitializationException` and the N+1 problem, the two most common real-world Hibernate bugs, with concrete fixes.

**You now understand not just Hibernate's syntax, but WHY it exists and the exact JDBC pain it removes — plus the two specific bugs (`LazyInitializationException`, N+1) that dominate real-world Hibernate debugging. This is the direct foundation for Spring Data JPA, covered later in this course, which builds an even higher-level, more convenient API on top of everything learned here.**

---

*(END OF HIBERNATE MASTER NOTES — ready to proceed to Spring Core whenever you say "Next Topic.")*